

Sumário

1. Objetivo Geral
 2. Arquitetura da Solução
 3. Estrutura de Projetos
 4. Módulo: CineTrack.Shared
 5. Módulo: CineTrack.API
 6. Módulo: CineTrack.Mobile
 7. Módulo: CineTrackRelayListener
 8. Módulo: CineTrackTokenGen
 9. Banco de Dados
 10. Roteiro de Implantação
 11. Configuração do Azure Relay
 12. Troubleshooting
-

1. Objetivo Geral

O que é o CineTrack?

O **CineTrack** é um aplicativo mobile pessoal para **gerenciamento de filmes e séries**. Ele permite que o usuário:

- **Acompanhe o que está assistindo** (filmes e séries em andamento)
- **Registre o que já assistiu** com notas e comentários pessoais
- **Visualize ratings de múltiplas fontes**: IMDb, Rotten Tomatoes (Critics e Audience), Metacritic
- **Acompanhe progresso de séries** episódio por episódio
- **Receba sugestões** baseadas em seu histórico
- **Sincronize dados** de qualquer lugar via Azure Relay

Por que o CineTrack foi criado?

O sistema resolve a necessidade de ter um **catálogo pessoal unificado** que:

1. **Não depende de serviços de streaming** - você controla seus dados
2. **Agrega ratings de várias fontes** em um só lugar
3. **Funciona offline** com sincronização quando conectado

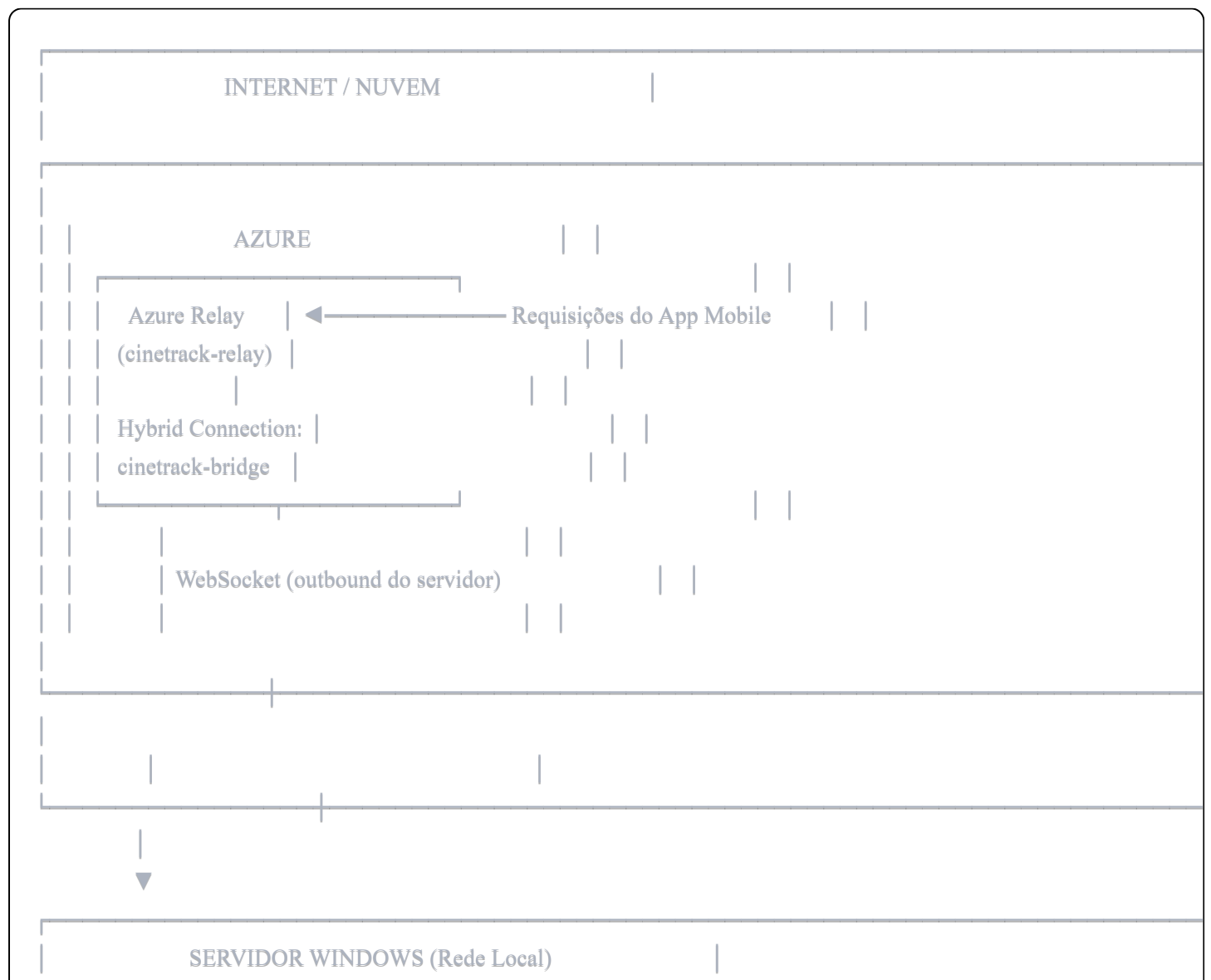
4. **Permite acesso remoto seguro** sem expor o banco de dados

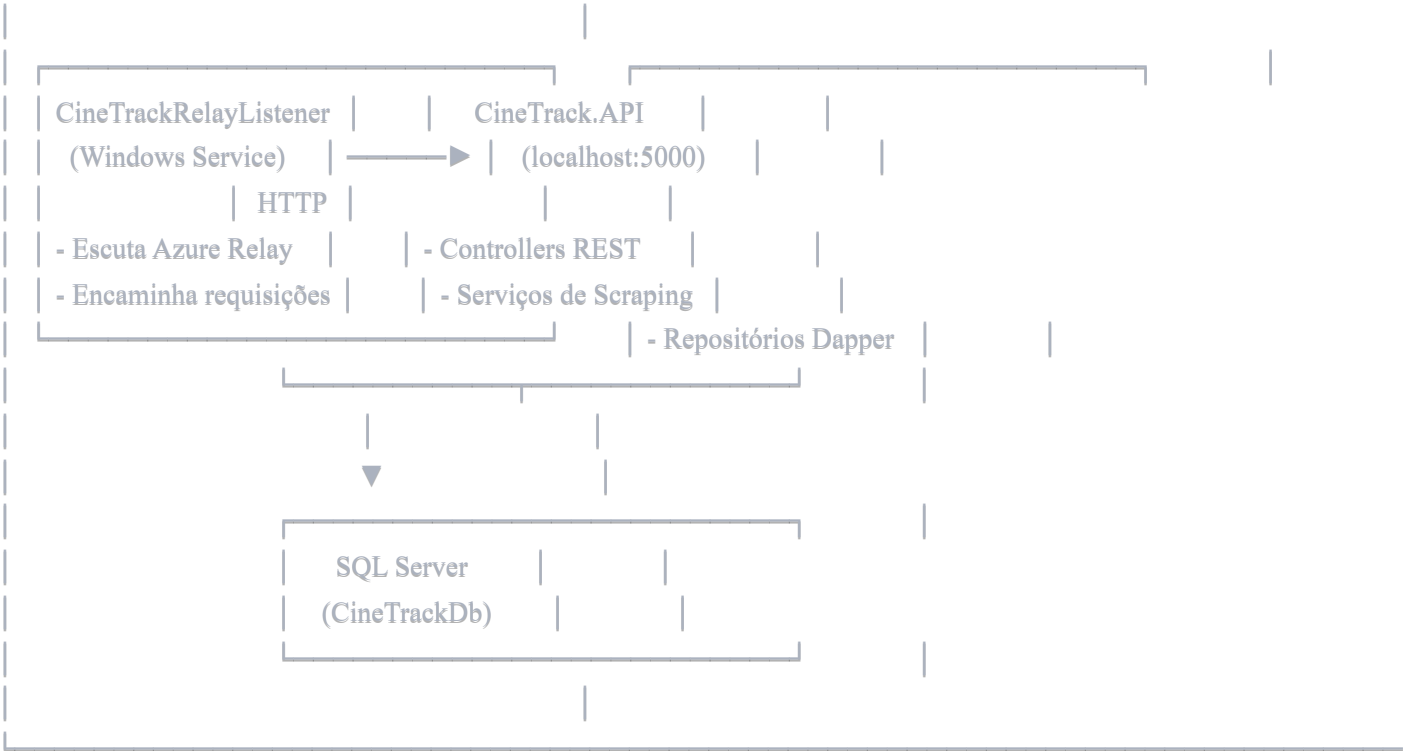
Diferenciais Técnicos

Característica	Benefício
Web Scraping Automatizado	Captura novos lançamentos de scnsrc.me automaticamente
Matching Inteligente	Usa algoritmos Levenshtein + Jaccard para associar títulos
Azure Relay	Acesso seguro ao servidor sem expor portas
4 Ratings Integrados	IMDb, RT Critics, RT Audience, Metacritic em uma tela
.NET MAUI Blazor	App nativo com interface web moderna

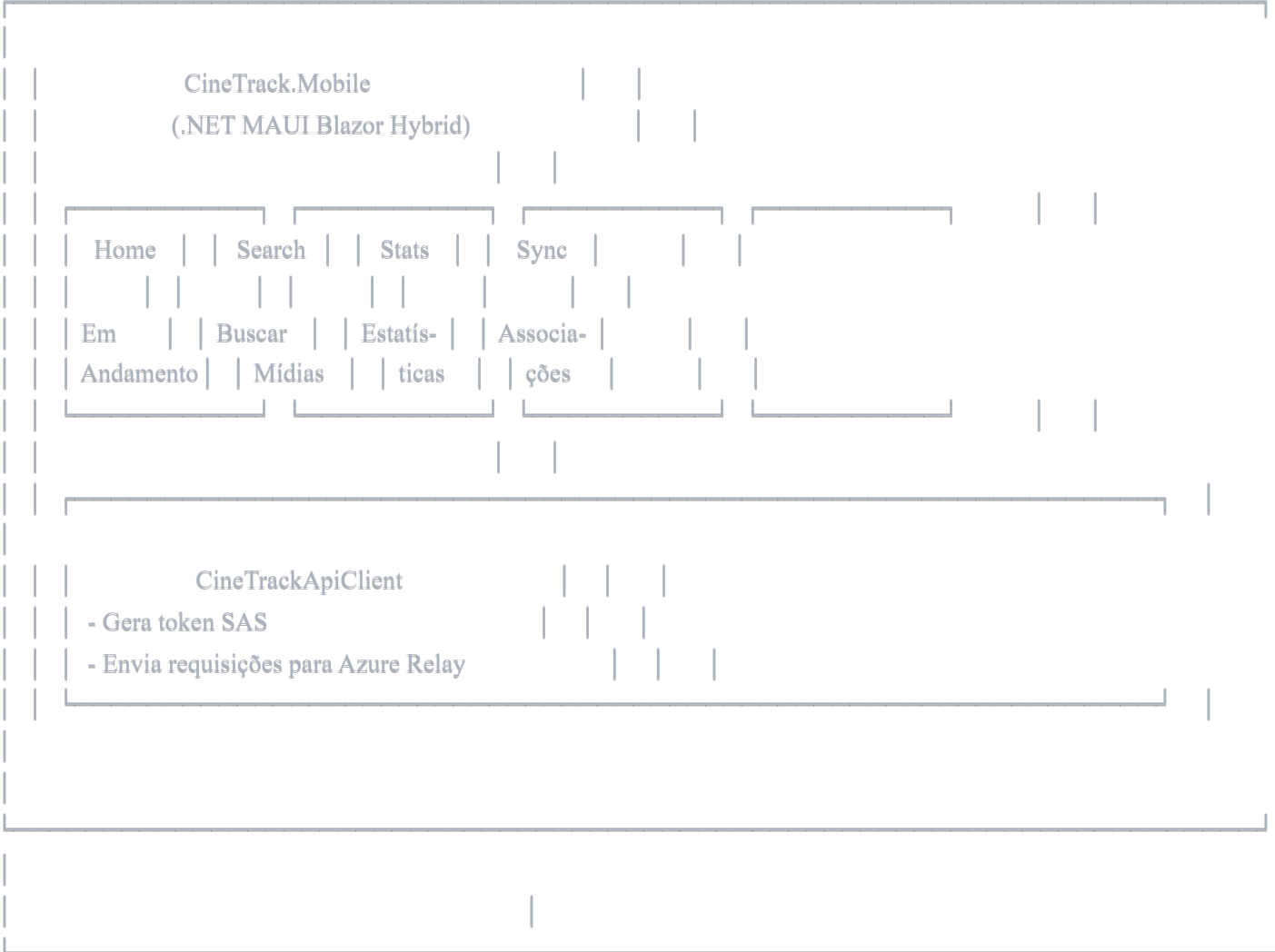
2. Arquitetura da Solução

Diagrama Geral

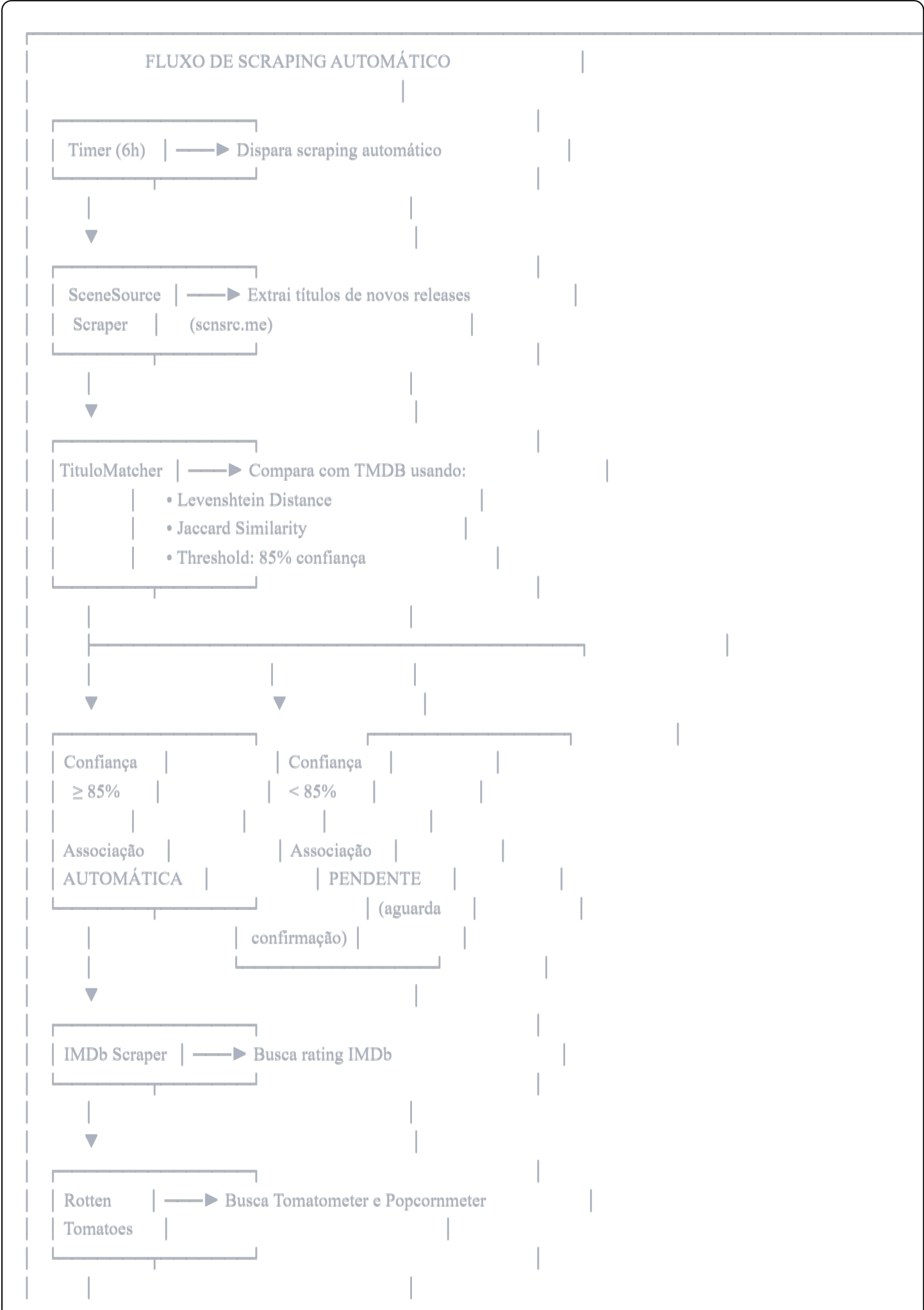




DISPOSITIVO MÓVEL



Fluxo de Dados do Scraping

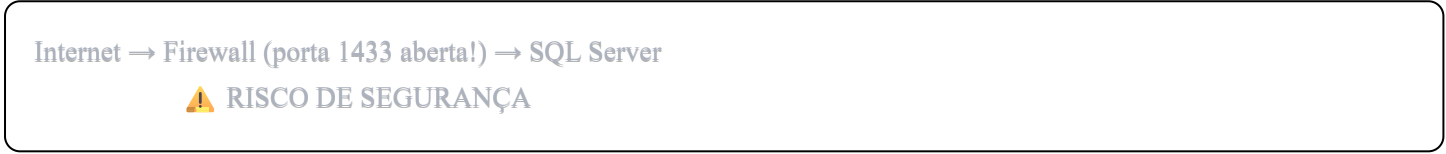




Por que Azure Relay?

O Azure Relay resolve um problema comum: **acessar um servidor em rede privada sem expor portas.**

Sem Azure Relay:

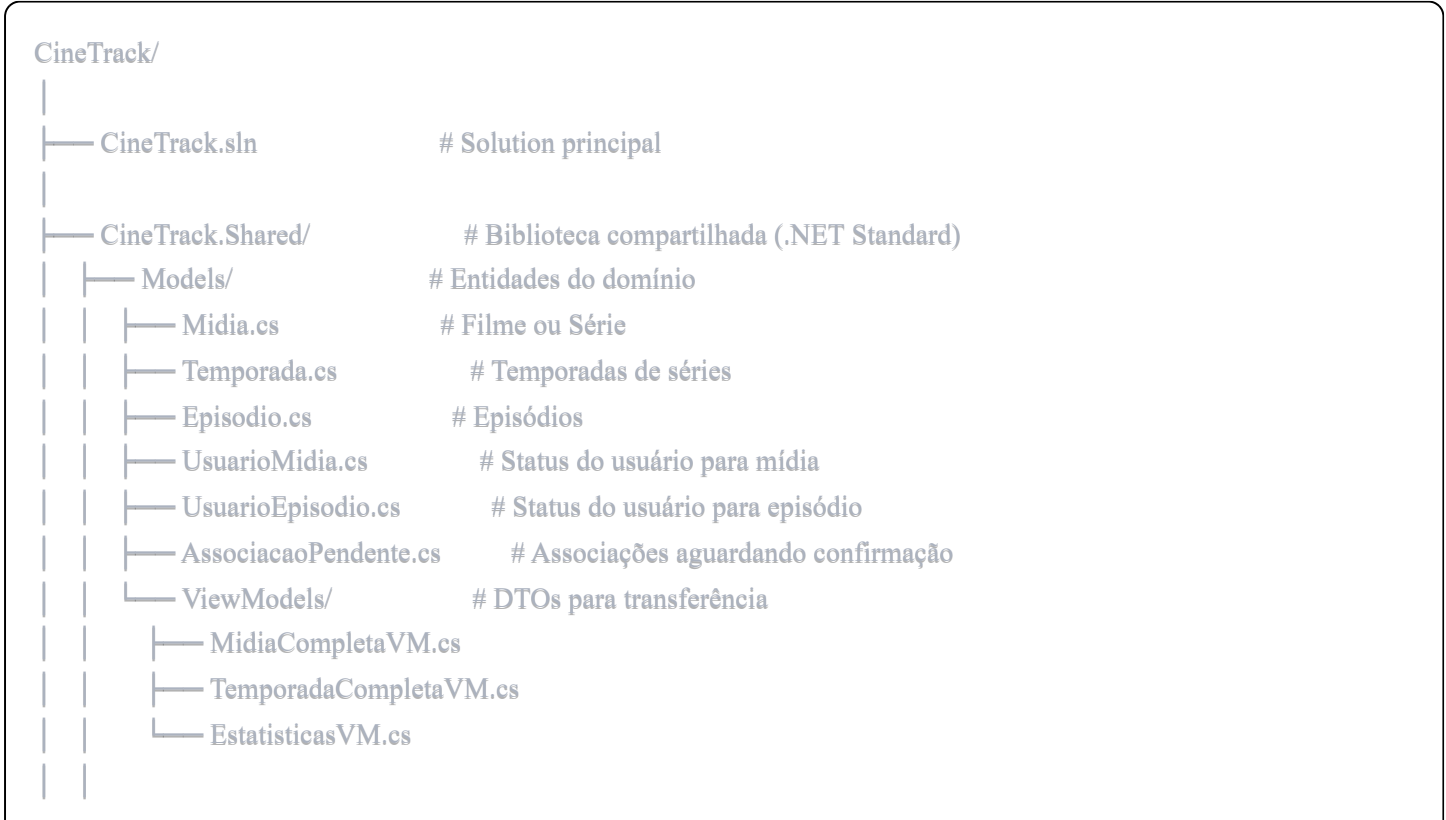


Com Azure Relay:

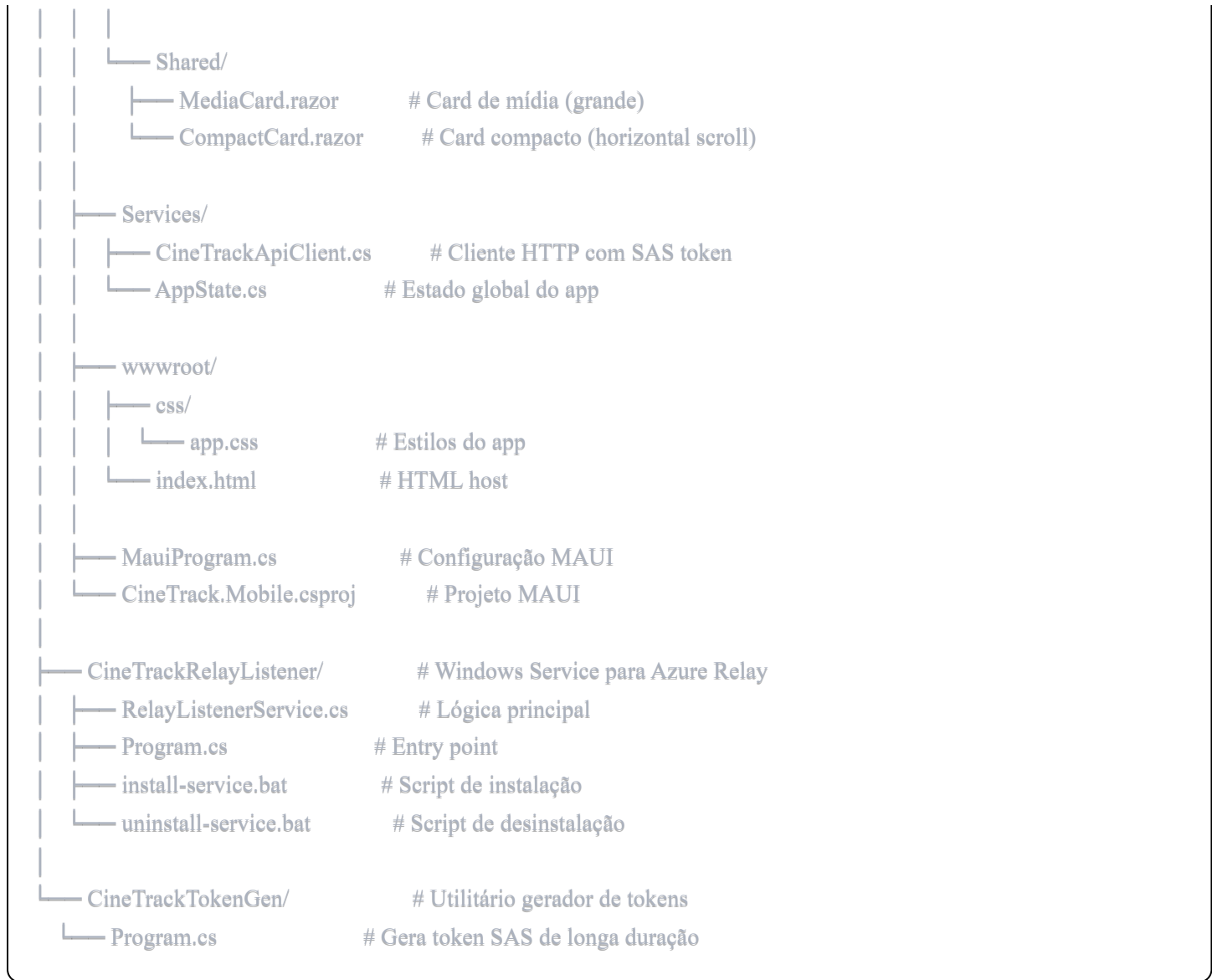


O **RelayListener** no servidor inicia uma conexão **de saída** (outbound) para o Azure. Como firewalls geralmente permitem conexões de saída, não é necessário abrir nenhuma porta.

3. Estrutura de Projetos



— Interfaces/	# Contratos
— IMidiaRepository.cs	
— ITemporadaRepository.cs	
— IEpisodioRepository.cs	
— IUsuarioMidiaRepository.cs	
— IScrapingService.cs	
— Data/	# Repositórios (Dapper)
— MidiaRepository.cs	
— TemporadaRepository.cs	
— EpisodioRepository.cs	
— UsuarioMidiaRepository.cs	
— Services/	# Serviços de negócio
— SceneSourceScraper.cs	# Scraper do sensrc.me
— TmdbService.cs	# Integração TMDb API
— ImdbScraper.cs	# Scraper do IMDb
— RottenTomatoesScraper.cs	# Scraper do Rotten Tomatoes
— TituloMatcher.cs	# Algoritmo de matching
— CineTrack.API/	# API REST (ASP.NET Core)
— Controllers/	
— MidiasController.cs	# CRUD de mídias
— TemporadasController.cs	# CRUD de temporadas
— EpisodiosController.cs	# CRUD de episódios
— ScrapingController.cs	# Endpoints de scraping
— SyncController.cs	# Sincronização
— Scraping/	# Serviços de captura
— ScrapingHostedService.cs	# Background service (timer)
— Models/	
— ScrapingModels.cs	# DTOs do scraping
— Program.cs	# Configuração e DI
— appsettings.json	# Connection strings e configs
— CineTrack.Mobile/	# App Mobile (.NET MAUI Blazor Hybrid)
— Components/	
— Layout/	
— MainLayout.razor	# Layout com bottom navigation
— Pages/	
— Home.razor	# Tela inicial
— Search.razor	# Busca de mídias
— Stats.razor	# Estatísticas
— Sync.razor	# Associações pendentes
— Detail.razor	# Detalhes da mídia



4. Módulo: CineTrack.Shared

Este módulo contém toda a lógica de negócio compartilhada entre API e Mobile.

4.1 Models

Midia.cs - Entidade Principal

csharp

```
// =====  
// Midia.cs  
// Representa um filme ou série no sistema  
// =====  
  
namespace CineTrack.Shared.Models;  
  
/// <summary>  
/// Entidade principal que representa um filme ou série.  
/// Armazena informações básicas, URLs de imagens e ratings de múltiplas fontes.  
/// </summary>  
public class Midia  
{  
    // =====  
    // IDENTIFICAÇÃO  
    // =====  
  
    /// <summary>Chave primária auto-incremento</summary>  
    public int Id { get; set; }  
  
    /// <summary>Título em português (exibido no app)</summary>  
    public string Titulo { get; set; } = "";  
  
    /// <summary>Título original (para buscas em APIs externas)</summary>  
    public string? TituloOriginal { get; set; }  
  
    /// <summary>Tipo: 1 = Filme, 2 = Série</summary>  
    public TipoMidia Tipo { get; set; }  
  
    /// <summary>Ano de lançamento</summary>  
    public int? Ano { get; set; }  
  
    // =====  
    // INFORMAÇÕES DESCRITIVAS  
    // =====  
  
    /// <summary>Gêneros separados por vírgula (ex: "Ação, Aventura")</summary>  
    public string? Generos { get; set; }  
  
    /// <summary>Sinopse da mídia</summary>  
    public string? Descricao { get; set; }  
  
    /// <summary>Duração em minutos (para filmes)</summary>  
    public int? DuracaoMinutos { get; set; }  
  
    /// <summary>Elenco principal em formato JSON</summary>
```



```
public string? Elenco { get; set; }
```

```
// =====  
// IMAGENS  
// =====
```

```
/// <summary>URL do poster (preferencialmente TMDB)</summary>  
public string? ImagemUrl { get; set; }
```

```
/// <summary>Imagem em Base64 para cache offline</summary>  
public string? ImagemBase64 { get; set; }
```

```
// =====  
// IDENTIFICADORES EXTERNOS  
// =====
```

```
/// <summary>ID do TMDB (ex: "550" para Fight Club)</summary>  
public int? TmdbId { get; set; }
```

```
/// <summary>ID do IMDb (ex: "tt0137523")</summary>  
public string? ImdbId { get; set; }
```

```
/// <summary>URL do Rotten Tomatoes</summary>  
public string? RottenTomatoesUrl { get; set; }
```

```
/// <summary>URL do SceneSource (origem do scraping)</summary>  
public string? SceneSourceUrl { get; set; }
```

```
// =====  
// RATINGS - 4 FONTES DIFERENTES  
// =====
```

```
/// <summary>Nota IMDb (0.0 a 10.0)</summary>  
public decimal? ImdbRating { get; set; }
```

```
/// <summary>Quantidade de votos no IMDb</summary>  
public int? ImdbVotes { get; set; }
```

```
/// <summary>Tomatometer - nota dos críticos (0 a 100)</summary>  
public int? Tomatometer { get; set; }
```

```
/// <summary>Popcornmeter - nota do público (0 a 100)</summary>  
public int? Popcornmeter { get; set; }
```

```
/// <summary>Nota do Metacritic (0 a 100)</summary>  
public int? Metacritic { get; set; }
```

```
// =====
// CONTROLE
// =====

/// <summary>Data de cadastro no sistema</summary>
public DateTime DataCadastro { get; set; }

/// <summary>Data da última atualização</summary>
public DateTime? DataAtualizacao { get; set; }

/// <summary>Status: 1 = Ativo, 0 = Inativo</summary>
public int Status { get; set; } = 1;

// =====
// NAVEGAÇÃO (Entity Framework)
// =====

/// <summary>Temporadas da série (vazio para filmes)</summary>
public List<Temporada> Temporadas { get; set; } = new();

/// <summary>Status do usuário para esta mídia</summary>
public UsuarioMidia? UsuarioMidia { get; set; }

// =====
// HELPERS (propriedades calculadas)
// =====

/// <summary>Retorna "Filme" ou "Série"</summary>
public string TipoDescricao => Tipo == TipoMidia.Filme ? "Filme" : "Série";

/// <summary>Formata gêneros com bullet points</summary>
public string? GenerosFormatados => Generos?.Replace(",", " • ");

/// <summary>Formata duração como "2h 15min"</summary>
public string DuracaoFormatada => DuracaoMinutos.HasValue
    ? $"{DuracaoMinutos / 60}h {DuracaoMinutos % 60}min"
    : "";
}

/// <summary>
/// Enum que define o tipo da mídia
/// </summary>
public enum TipoMidia
{
    Filme = 1,
```

Serie = 2

}

UsuarioMidia.cs - Status do Usuário

csharp

```
// =====
// UsuarioMidia.cs
// Status do usuário para uma mídia específica
// =====

namespace CineTrack.Shared.Models;

/// <summary>
/// Registra o status de uma mídia para o usuário:
/// - Se está assistindo, já assistiu, pendente ou desistiu
/// - Nota pessoal e comentários
/// - Com quem está assistindo
/// </summary>
public class UsuarioMidia
{
    public int Id { get; set; }
    public int MidiaId { get; set; }

    /// <summary>Status atual: Assistindo, Assistido, Pendente, Desistência</summary>
    public StatusMidia Status { get; set; }

    /// <summary>Com quem está assistindo (sozinho ou acompanhado)</summary>
    public AssistindoCom AssistindoCom { get; set; }

    /// <summary>Nota pessoal do usuário (0.0 a 10.0)</summary>
    public decimal? NotaUsuario { get; set; }

    /// <summary>Avaliação em estrelas (1 a 5)</summary>
    public int? EstrelaUsuario { get; set; }

    /// <summary>Comentário/review pessoal</summary>
    public string? Comentario { get; set; }

    /// <summary>Quando começou a assistir</summary>
    public DateTime? DataInicio { get; set; }

    /// <summary>Quando terminou de assistir</summary>
    public DateTime? DataConclusao { get; set; }

    public DateTime DataCadastro { get; set; }
    public DateTime? DataAtualizacao { get; set; }

    // =====
    // HELPERS
    // =====
```

```

public string StatusDescricao => Status switch
{
    StatusMidia.Assistindo => "Assistindo",
    StatusMidia.Assistido => "Assistido",
    StatusMidia.Pendente => "Pendente",
    StatusMidia.Desistencia => "Desistência",
    _ => "Desconhecido"
};

```

/// <summary>Cor do badge de status para UI</summary>

```

public string CorStatus => Status switch
{
    StatusMidia.Assistindo => "#3b82f6", // Azul
    StatusMidia.Assistido => "#22c55e", // Verde
    StatusMidia.Pendente => "#f59e0b", // Amarelo
    StatusMidia.Desistencia => "#ef4444", // Vermelho
    _ => "#6b7280" // Cinza
};
}

```

```

public enum StatusMidia

```

```

{
    Assistindo = 1,
    Assistido = 2,
    Pendente = 3,
    Desistencia = 4
}

```

```

public enum AssistindoCom

```

```

{
    Sozinho = 1,
    Andria = 2 // Personalizável
}

```

4.2 Services - TituloMatcher.cs

csharp

```
// =====  
// TituloMatcher.cs  
// Algoritmo de matching de títulos usando múltiplas métricas  
// =====  
  
using System.Text.RegularExpressions;  
  
namespace CineTrack.Shared.Services;  
  
/// <summary>  
/// Serviço responsável por comparar títulos e determinar se são a mesma mídia.  
/// Usa combinação de algoritmos para maior precisão:  
/// - Levenshtein Distance: mede edições necessárias para transformar uma string em outra  
/// - Jaccard Similarity: compara conjuntos de palavras  
/// </summary>  
public class TituloMatcher  
{  
    // =====  
    // CONFIGURAÇÕES  
    // =====  
  
    /// <summary>  
    /// Threshold mínimo para considerar uma associação automática.  
    /// Valores abaixo disso vão para revisão manual.  
    /// </summary>  
    private const double THRESHOLD_AUTO = 0.85; // 85%  
  
    /// <summary>  
    /// Threshold mínimo para considerar uma associação possível.  
    /// Abaixo disso é descartado.  
    /// </summary>  
    private const double THRESHOLD_MIN = 0.60; // 60%  
  
    // =====  
    // MÉTODO PRINCIPAL  
    // =====  
  
    /// <summary>  
    /// Compara dois títulos e retorna o grau de similaridade (0,0 a 1,0).  
    /// </summary>  
    /// <param name="titulo1">Primeiro título (geralmente do SceneSource)</param>  
    /// <param name="titulo2">Segundo título (geralmente do TMDb)</param>  
    /// <returns>  
    /// Tupla com:  
    /// - Score: valor entre 0,0 e 1,0  
    /// - IsAutoMatch: true se score >= 85%
```

```
/// - IsPossibleMatch: true se score >= 60%
```

```
/// </returns>
```

```
public (double Score, bool IsAutoMatch, bool IsPossibleMatch) Compare(
```

```
    string titulo1,
```

```
    string titulo2)
```

```
{
```

```
    // Normaliza os títulos para comparação justa
```

```
    var t1 = Normalize(titulo1);
```

```
    var t2 = Normalize(titulo2);
```

```
    // Se após normalização são iguais, é match perfeito
```

```
    if (t1 == t2)
```

```
        return (1.0, true, true);
```

```
    // Calcula similaridade usando dois algoritmos
```

```
    // Levenshtein é bom para pequenas diferenças (typos)
```

```
    // Jaccard é bom para reordenação de palavras
```

```
    var levenshtein = CalculateLevenshteinSimilarity(t1, t2);
```

```
    var jaccard = CalculateJaccardSimilarity(t1, t2);
```

```
    // Usa a média ponderada (Levenshtein tem mais peso)
```

```
    // pois é mais preciso para títulos similares
```

```
    var score = (levenshtein * 0.6) + (jaccard * 0.4);
```

```
    return (
```

```
        Score: Math.Round(score, 4),
```

```
        IsAutoMatch: score >= THRESHOLD_AUTO,
```

```
        IsPossibleMatch: score >= THRESHOLD_MIN
```

```
    );
```

```
}
```

```
/// =====
```

```
/// NORMALIZAÇÃO
```

```
/// =====
```

```
/// <summary>
```

```
/// Normaliza um título para comparação:
```

```
/// - Converte para minúsculas
```

```
/// - Remove acentos
```

```
/// - Remove caracteres especiais
```

```
/// - Remove artigos comuns (The, A, O, Os, As)
```

```
/// - Remove indicadores de qualidade (720p, 1080p, BluRay, etc)
```

```
/// </summary>
```

```
private string Normalize(string input)
```

```
{
```

```
    if (string.IsNullOrEmpty(input))
```

```
        return "";
```

```

var result = input.ToLowerInvariant();

// Remove acentos
result = RemoveAccents(result);

// Remove indicadores de qualidade/release
result = Regex.Replace(result,
    @"(720p|1080p|2160p|4k|bluray|web-dl|webrip|hdrip|brrip|x264|x265|hevc|aac|ac3|dts|hdv|dvdrip|proper|repack|exten
    """,
    RegexOptions.IgnoreCase);

// Remove ano entre parênteses ou após o título
result = Regex.Replace(result, @"\"(?\d{4})\"?", "");

// Remove indicadores de temporada/episódio
result = Regex.Replace(result, @"s\d{1,2}e\d{1,2}", "", RegexOptions.IgnoreCase);

// Remove artigos comuns
result = Regex.Replace(result, @"\"b(the|a|an|o|os|a|as|um|uma|uns|umas)\"b", "");

// Remove caracteres especiais, mantém apenas letras e números
result = Regex.Replace(result, @"\"[^a-z0-9\s]\"", "");

// Remove espaços múltiplos e trim
result = Regex.Replace(result, @"\"s+\"", " ").Trim();

return result;
}

// =====
// ALGORITMO LEVENSHTein
// =====

/// <summary>
/// Calcula a similaridade baseada na distância de Levenshtein.
/// A distância de Levenshtein é o número mínimo de operações
/// (inserção, remoção, substituição) para transformar uma string em outra.
/// </summary>
private double CalculateLevenshteinSimilarity(string s1, string s2)
{
    if (string.IsNullOrEmpty(s1) && string.IsNullOrEmpty(s2))
        return 1.0;
    if (string.IsNullOrEmpty(s1) || string.IsNullOrEmpty(s2))
        return 0.0;

    var distance = LevenshteinDistance(s1, s2);

```



```

var maxLength = Math.Max(s1.Length, s2.Length);

// Converte distância em similaridade (0 a 1)
return 1.0 - ((double)distance / maxLength);
}

/// <summary>
/// Implementação do algoritmo de Levenshtein usando programação dinâmica.
/// Complexidade:  $O(m*n)$  onde  $m$  e  $n$  são os tamanhos das strings.
/// </summary>
private int LevenshteinDistance(string s1, string s2)
{
    var m = s1.Length;
    var n = s2.Length;

    // Matriz de distâncias
    var d = new int[m + 1, n + 1];

    // Inicializa primeira coluna e linha
    for (var i = 0; i <= m; i++) d[i, 0] = i;
    for (var j = 0; j <= n; j++) d[0, j] = j;

    // Preenche a matriz
    for (var i = 1; i <= m; i++)
    {
        for (var j = 1; j <= n; j++)
        {
            // Custo de substituição (0 se caracteres iguais)
            var cost = s1[i - 1] == s2[j - 1] ? 0 : 1;

            // Mínimo entre: remoção, inserção, substituição
            d[i, j] = Math.Min(
                Math.Min(
                    d[i - 1, j] + 1,    // Remoção
                    d[i, j - 1] + 1),  // Inserção
                d[i - 1, j - 1] + cost); // Substituição
        }
    }

    return d[m, n];
}

// =====
// ALGORITMO JACCARD
// =====

/// <summary>

```

```
/// Calcula a similaridade de Jaccard entre dois conjuntos de palavras.
```

```
/// Jaccard =  $|A \cap B| / |A \cup B|$ 
```

```
/// Ou seja: palavras em comum / total de palavras únicas
```

```
/// </summary>
```

```
private double CalculateJaccardSimilarity(string s1, string s2)
```

```
{
```

```
    // Divide em palavras (conjunto)
```

```
    var set1 = new HashSet<string>(s1.Split(' ', StringSplitOptions.RemoveEmptyEntries));
```

```
    var set2 = new HashSet<string>(s2.Split(' ', StringSplitOptions.RemoveEmptyEntries));
```

```
    if (set1.Count == 0 && set2.Count == 0)
```

```
        return 1.0;
```

```
    if (set1.Count == 0 || set2.Count == 0)
```

```
        return 0.0;
```

```
    // Calcula interseção
```

```
    var intersection = new HashSet<string>(set1);
```

```
    intersection.IntersectWith(set2);
```

```
    // Calcula união
```

```
    var union = new HashSet<string>(set1);
```

```
    union.UnionWith(set2);
```

```
    return (double)intersection.Count / union.Count;
```

```
}
```

```
/// =====
```

```
/// HELPERS PARA PARSING
```

```
/// =====
```

```
/// <summary>
```

```
/// Extrai informações estruturadas de um nome de release.
```

```
/// Exemplo: "Breaking.Bad.S01E01.720p.BluRay.x264"
```

```
/// Retorna: (Titulo: "Breaking Bad", Ano: null, Temporada: 1, Episodio: 1)
```

```
/// </summary>
```

```
public (string Titulo, int? Ano, int? Temporada, int? Episodio) ParseTitle(string rawTitle)
```

```
{
```

```
    var titulo = rawTitle;
```

```
    int? ano = null;
```

```
    int? temporada = null;
```

```
    int? episodio = null;
```

```
    // Extrai ano (4 dígitos entre 1900-2099)
```

```
    var anoMatch = Regex.Match(rawTitle, @"\"b(19|20)\d{2}\"b");
```

```
    if (anoMatch.Success)
```

```
    {
```

```
        ano = int.Parse(anoMatch.Value);
```

```

    }

    // Extrai temporada/episódio (S01E01)
    var seMatch = Regex.Match(rawTitle, @"S\d{1,2}E\d{1,2}", RegexOptions.IgnoreCase);
    if (seMatch.Success)
    {
        temporada = int.Parse(seMatch.Groups[1].Value);
        episodio = int.Parse(seMatch.Groups[2].Value);
    }

    // Limpa título
    titulo = Regex.Replace(rawTitle, @"S\d{1,2}E\d{1,2}", "", RegexOptions.IgnoreCase);
    titulo = Regex.Replace(titulo, @"b(19|20)\d{2}b", "");
    titulo = Regex.Replace(titulo,
        @"(720p|1080p|2160p|4K|BluRay|WEB-DL|WEBRip|HDRip|BRRip|x264|x265|HEVC|AAC|AC3|DTS).*",
        "",
        RegexOptions.IgnoreCase);
    titulo = Regex.Replace(titulo, @"[\.\-_\]", " ");
    titulo = Regex.Replace(titulo, @"\s+", " ").Trim();

    return (titulo, ano, temporada, episodio);
}

private string RemoveAccents(string input)
{
    var normalized = input.Normalize(System.Text.NormalizationForm.FormD);
    var builder = new System.Text.StringBuilder();

    foreach (var c in normalized)
    {
        if (System.Globalization.CharUnicodeInfo.GetUnicodeCategory(c)
            != System.Globalization.UnicodeCategory.NonSpacingMark)
        {
            builder.Append(c);
        }
    }

    return builder.ToString().Normalize(System.Text.NormalizationForm.FormC);
}
}

```

5. Módulo: CineTrack.API

A API REST que expõe os dados para o aplicativo mobile.

5.1 Program.cs - Configuração

csharp

```
// =====  
// Program.cs  
// Configuração e inicialização da API  
// =====  
  
using CineTrack.Shared.Data;  
using CineTrack.Shared.Interfaces;  
using CineTrack.Shared.Services;  
using CineTrack.API.Scraping;  
  
var builder = WebApplication.CreateBuilder(args);  
  
// =====  
// CONFIGURAÇÃO DE SERVIÇOS  
// =====  
  
// Controllers  
builder.Services.AddControllers();  
  
// Swagger (documentação da API)  
builder.Services.AddEndpointsApiExplorer();  
builder.Services.AddSwaggerGen();  
  
// String de conexão com o SQL Server  
var connectionString = builder.Configuration.GetConnectionString("DefaultConnection");  
  
// Registra repositórios com injeção de dependência  
builder.Services.AddScoped<IMidiaRepository>(sp =>  
    new MidiaRepository(connectionString!));  
builder.Services.AddScoped<ITemporadaRepository>(sp =>  
    new TemporadaRepository(connectionString!));  
builder.Services.AddScoped<IEpisodioRepository>(sp =>  
    new EpisodioRepository(connectionString!));  
builder.Services.AddScoped<IUsuarioMidiaRepository>(sp =>  
    new UsuarioMidiaRepository(connectionString!));  
  
// HttpClient para scrapers  
builder.Services.AddHttpClient<ISceneSourceScraper, SceneSourceScraper>();  
builder.Services.AddHttpClient<ITmdbService, TmdbService>();  
builder.Services.AddHttpClient<IImdbScraper, ImdbScraper>();  
builder.Services.AddHttpClient<IRottenTomatoesScraper, RottenTomatoesScraper>();  
  
// Serviço de matching  
builder.Services.AddSingleton<TituloMatcher>();  
  
// Background service para scraping automático
```

```

if (builder.Configuration.GetValue<bool>("Scraping:Enabled"))
{
    builder.Services.AddHostedService<ScrapingHostedService>();
}

// =====
// BUILD E MIDDLEWARE
// =====

var app = builder.Build();

// Swagger apenas em desenvolvimento
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseAuthorization();
app.MapControllers();

// Health check endpoint
app.MapGet("/health", () => Results.Ok(new {
    Status = "Healthy",
    Time = DateTime.UtcNow
}));

app.Run();

```

5.2 appsettings.json

json

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=localhost;Database=CineTrackDb;Trusted_Connection=True;TrustServerCertificate=True;"
  },
  "Scraping": {
    "Enabled": true,
    "IntervalHours": 6,
    "TmdbApiKey": "SUA-API-KEY-TMDB-AQUI"
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*"
}
```

6. Módulo: CineTrack.Mobile

Aplicativo mobile desenvolvido em .NET MAUI Blazor Hybrid.

6.1 CineTrackApiClient.cs - Cliente HTTP com SAS

csharp

```
// =====
// CineTrackApiClient.cs
// Cliente HTTP que se comunica via Azure Relay
// =====

using CineTrack.Shared.Models;
using System.Net.Http.Json;
using System.Security.Cryptography;
using System.Text;
using System.Text.Json;
using System.Web;

namespace CineTrack.Mobile.Services;

/// <summary>
/// Cliente HTTP para comunicação com a API CineTrack via Azure Relay.
///
/// O Azure Relay requer autenticação via token SAS (Shared Access Signature).
/// Este cliente gera o token automaticamente e o inclui em cada requisição.
/// </summary>
public class CineTrackApiClient
{
    // =====
    // CONFIGURAÇÃO DO AZURE RELAY
    // =====

    /// <summary>Namespace do Relay (criado no Azure)</summary>
    private const string RelayNamespace = "cinetrack-relay";

    /// <summary>Nome da Hybrid Connection</summary>
    private const string HybridConnectionName = "cinetrack-bridge";

    /// <summary>Nome da política SAS</summary>
    private const string SasKeyName = "ListenSend";

    /// <summary>
    /// Chave SAS (obtida no Portal Azure)
    /// ATENÇÃO: Em produção, esta chave deveria estar em local seguro
    /// </summary>
    private const string SasKey = "SUA-CHAVE-SAS-AQUI";

    /// <summary>
    /// Token SAS pré-gerado (alternativa ao SasKey)
    /// Use o CineTrackTokenGen para gerar um token de longa duração
    /// </summary>
    private const string SasToken = ""; // Se preenchido, usa este
```



```
/// <summary>URL base para requisições</summary>
```

```
private const string BaseUrl = "https://cinetrack-relay.servicebus.windows.net/cinetrack-bridge";
```

```
private readonly HttpClient _httpClient;
```

```
private readonly JsonSerializerOptions _jsonOptions;
```

```
public CineTrackApiClient()
```

```
{  
    _httpClient = new HttpClient  
    {  
        BaseAddress = new Uri(BaseUrl),  
        Timeout = TimeSpan.FromSeconds(30)  
    };  
  
    _jsonOptions = new JsonSerializerOptions  
    {  
        PropertyNameCaseInsensitive = true,  
        PropertyNamingPolicy = JsonNamingPolicy.CamelCase  
    };  
}
```

```
// =====  
// MÉTODOS DE API  
// =====
```

```
public async Task<List<MidiaCompletaVM>> GetMidiasAsync(  
    StatusMidia? status = null,  
    TipoMidia? tipo = null)
```

```
{  
    var query = new List<string>();  
    if (status.HasValue) query.Add($"status={{(int)status}}");  
    if (tipo.HasValue) query.Add($"tipo={{(int)tipo}}");  
  
    var url = "/api/midias";  
    if (query.Any()) url += "?" + string.Join("&", query);  
  
    return await GetAsync<List<MidiaCompletaVM>>(url) ?? new();  
}
```

```
public async Task<MidiaCompletaVM?> GetMidiaAsync(int id)
```

```
{  
    return await GetAsync<MidiaCompletaVM>($"/api/midias/{id}");  
}
```

```
public async Task<List<MidiaCompletaVM>> GetEmAndamentoAsync()
```

```
{
```

```

        return await GetAsync<List<MidiaCompletaVM>>("/api/midias/emandamento") ?? new();
    }

    public async Task<List<MidiaCompletaVM>> GetRecentesAsync(int quantidade = 10)
    {
        return await GetAsync<List<MidiaCompletaVM>>($"api/midias/recentes?quantidade={quantidade}") ?? new();
    }

    public async Task<bool> HealthCheckAsync()
    {
        try
        {
            var result = await GetAsync<object>("/health");
            return result != null;
        }
        catch
        {
            return false;
        }
    }
}

```

```

// =====
// MÉTODOS HTTP GENÉRICOS
// =====

```

```

private async Task<T?> GetAsync<T>(string endpoint)
{
    var request = new HttpRequestMessage(HttpMethod.Get, endpoint);
    AddSasToken(request);

    var response = await _httpClient.SendAsync(request);
    response.EnsureSuccessStatusCode();

    var content = await response.Content.ReadAsStringAsync();
    return JsonSerializer.Deserialize<T>(content, _jsonOptions);
}

```

```

// =====
// AUTENTICAÇÃO SAS
// =====

```

```

/// <summary>
/// Adiciona header de autenticação SAS à requisição
/// </summary>
private void AddSasToken(HttpRequestMessage request)
{
    string token;

```

```

if (!string.IsNullOrEmpty(SasToken))
{
    // Usa token pré-gerado
    token = SasToken;
}
else
{
    // Gera token dinamicamente
    token = CreateSasToken(
        $"https://{RelayNamespace}.servicebus.windows.net/{HybridConnectionName}",
        SasKeyName,
        SasKey,
        TimeSpan.FromHours(1));
}

// Header específico do Azure Relay
request.Headers.Add("ServiceBusAuthorization", token);
}

/// <summary>
/// Gera um token SAS (Shared Access Signature)
///
/// O token SAS é composto por:
/// - sr: Resource URI (URL do Relay)
/// - sig: Signature (HMAC-SHA256 do recurso + expiração)
/// - se: Expiration (timestamp Unix)
/// - skn: Key Name (nome da política)
/// </summary>
private static string CreateSasToken(
    string resourceUri,
    string keyName,
    string key,
    TimeSpan validity)
{
    // Calcula timestamp de expiração
    var sinceEpoch = DateTime.UtcNow - new DateTime(1970, 1, 1);
    var expiry = Convert.ToString((int)sinceEpoch.TotalSeconds + validity.TotalSeconds);

    // String a ser assinada: URL encoded do recurso + \n + expiração
    var stringToSign = HttpUtility.UrlEncode(resourceUri) + "\n" + expiry;

    // Calcula HMAC-SHA256
    using var hmac = new HMACSHA256(Encoding.UTF8.GetBytes(key));
    var signature = Convert.ToBase64String(
        hmac.ComputeHash(Encoding.UTF8.GetBytes(stringToSign)));
}

```

```
// Monta o token
```

```
var token = string.Format(  
    "SharedAccessSignature sr={0}&sig={1}&se={2}&skn={3}",  
    HttpUtility.UrlEncode(resourceUri),  
    HttpUtility.UrlEncode(signature),  
    expiry,  
    keyName);  
  
return token;  
}  
}
```

7. Módulo: CineTrackRelayListener

Serviço Windows que escuta o Azure Relay e encaminha requisições para a API local.

7.1 RelayListenerService.cs

```
csharp
```

```
// =====
// RelayCommandService.cs
// Windows Service que conecta ao Azure Relay
// =====

using Microsoft.Azure.Relay;
using System.Net;

namespace CineTrackRelayListener;

/// <summary>
/// Serviço que escuta requisições do Azure Relay e as encaminha para a API local.
///
/// FUNCIONAMENTO:
/// 1. Conecta ao Azure Relay via WebSocket (conexão de saída)
/// 2. Aguarda requisições HTTP chegarem via Relay
/// 3. Encaminha cada requisição para a API local (localhost:5000)
/// 4. Retorna a resposta da API para o cliente original
/// </summary>
public class RelayCommandService : BackgroundService
{
    // =====
    // CONFIGURAÇÃO - PREENCHA COM SEUS DADOS
    // =====

    /// <summary>
    /// Connection string do Azure Relay.
    /// Obtida no Portal Azure > Relay > Hybrid Connections > Shared access policies
    /// </summary>
    private const string RelayConnectionString =
        "Endpoint=sb://cinetrack-relay.servicebus.windows.net/;" +
        "SharedAccessKeyName=ListenSend;" +
        "SharedAccessKey=SUA-CHAVE-AQUI;" +
        "EntityPath=cinetrack-bridge";

    /// <summary>Nome da Hybrid Connection (deve coincidir com EntityPath)</summary>
    private const string HybridConnectionName = "cinetrack-bridge";

    /// <summary>URL da API local que receberá as requisições</summary>
    private const string CineTrackApiBaseUrl = "http://localhost:5000";

    private readonly ILogger<RelayListenerService> _logger;
    private HybridConnectionListener? _listener;
    private readonly HttpClient _httpClient;

    public RelayCommandService(ILogger<RelayListenerService> logger)
```

```

{
    _logger = logger;
    _httpClient = new HttpClient
    {
        BaseAddress = new Uri(CineTrackApiBaseUrl),
        Timeout = TimeSpan.FromSeconds(60)
    };
}

protected override async Task ExecuteAsync(CancellationToken stoppingToken)
{
    _logger.LogInformation("=====");
    _logger.LogInformation("CineTrack Relay Listener iniciando...");
    _logger.LogInformation("API Local: {Url}", CineTrackApiBaseUrl);
    _logger.LogInformation("=====");

    try
    {
        // Cria o listener do Relay
        _listener = new HybridConnectionListener(RelayConnectionString);

        // Handler para requisições HTTP
        _listener.RequestHandler = ProcessRequestAsync;

        // Abre conexão com o Azure
        await _listener.OpenAsync(stoppingToken);
        _logger.LogInformation("✅ Conectado ao Azure Relay!");
        _logger.LogInformation("Aguardando requisições...");

        // Mantém o serviço rodando até ser cancelado
        while (!stoppingToken.IsCancellationRequested)
        {
            await Task.Delay(1000, stoppingToken);
        }
    }
    catch (Exception ex)
    {
        _logger.LogError(ex, "❌ Erro fatal no Relay Listener");
        throw;
    }
}

/// <summary>
/// Processa cada requisição recebida do Azure Relay
/// </summary>
private async void ProcessRequestAsync(RelayedHttpListenerContext context)
{

```

```
var request = context.Request;
var response = context.Response;

try
{
    var path = request.Url.PathAndQuery;
    var method = new HttpMethod(request.HttpMethod);

    // IMPORTANTE: Remove o prefixo do Hybrid Connection do path
    if (path.StartsWith("/") + HybridConnectionName)
    {
        path = path.Substring(("/" + HybridConnectionName).Length);
    }
    if (string.IsNullOrEmpty(path))
    {
        path = "/";
    }

    _logger.LogInformation("[{Method}] {Path}", method, path);

    // Cria requisição para a API local
    var localRequest = new HttpRequestMessage(method, path);

    // Copia headers relevantes
    foreach (var header in request.Headers.AllKeys)
    {
        if (header.Equals("Host", StringComparison.OrdinalIgnoreCase) ||
            header.Equals("Content-Length", StringComparison.OrdinalIgnoreCase) ||
            header.StartsWith("ServiceBus", StringComparison.OrdinalIgnoreCase))
        {
            continue;
        }
        localRequest.Headers.TryAddWithoutValidation(header, request.Headers[header]);
    }

    // Copia corpo da requisição (para POST/PUT)
    if (request.HasEntityBody)
    {
        using var reader = new StreamReader(request.InputStream);
        var body = await reader.ReadToEndAsync();
        localRequest.Content = new StringContent(
            body,
            System.Text.Encoding.UTF8,
            request.ContentType ?? "application/json");
    }

    // Envia para a API local
```

```

var localResponse = await _httpClient.SendAsync(localRequest);

// Cópia resposta
response.StatusCode = localResponse.StatusCode;
var responseBody = await localResponse.Content.ReadAsStringAsync();
var responseBytes = System.Text.Encoding.UTF8.GetBytes(responseBody);
await response.OutputStream.WriteAsync(responseBytes);

_logger.LogInformation("[{Method}] {Path} → {Status}",
    method, path, (int)localResponse.StatusCode);
}
catch (Exception ex)
{
    _logger.LogError(ex, "Erro ao processar requisição");
    response.StatusCode = HttpStatusCode.InternalServerError;
    var errorBytes = System.Text.Encoding.UTF8.GetBytes(
        $"{{\"error\": \"{ex.Message}\"}}");
    await response.OutputStream.WriteAsync(errorBytes);
}
finally
{
    response.Close();
}
}
}

```

7.2 install-service.bat

batch

@echo off

REM =====

REM install-service.bat

REM Instala o CineTrackRelayListener como Windows Service

REM Execute como Administrador!

REM =====

echo.

echo =====

echo CineTrack Relay Listener - Instalacao

echo =====

echo.

REM Verifica se está rodando como Admin

net session >nul 2>&1

if %errorLevel% neq 0 (

 echo ERRO: Execute como Administrador!

 pause

 exit /b 1

)

REM Para o serviço se já existir

echo Parando servico existente...

sc stop CineTrackRelayListener >nul 2>&1

timeout /t 2 >nul

REM Remove serviço antigo

echo Removendo servico antigo...

sc delete CineTrackRelayListener >nul 2>&1

timeout /t 2 >nul

REM Instala novo serviço

echo Instalando servico...

sc create CineTrackRelayListener ^

 binPath= "%~dp0CineTrackRelayListener.exe" ^

 start= auto ^

 DisplayName= "CineTrack Relay Listener"

REM Configura descrição

sc description CineTrackRelayListener "Escuta Azure Relay e encaminha requisicoes para a API CineTrack local"

REM Inicia o serviço

echo Iniciando servico...

sc start CineTrackRelayListener

echo.

```
echo =====  
echo Servico instalado e iniciado!  
echo =====  
echo.  
echo Comandos uteis:  
echo  sc query CineTrackRelayListener - Ver status  
echo  sc stop CineTrackRelayListener - Parar  
echo  sc delete CineTrackRelayListener - Remover  
echo.  
pause
```

8. Módulo: CineTrackTokenGen

Utilitário para gerar tokens SAS de longa duração.

8.1 Program.cs

```
csharp
```

```
// =====
// CineTrackTokenGen/Program.cs
// Gerador de tokens SAS para Azure Relay
// =====

using Microsoft.Azure.Relay;

// =====
// CONFIGURAÇÃO - PREENCHA COM SEUS DADOS
// =====

var keyName = "ListenSend";
var key = "SUA-CHAVE-SAS-AQUI";
var uri = "https://cinetrack-relay.servicebus.windows.net/cinetrack-bridge";

// =====
// GERAÇÃO DO TOKEN
// =====

Console.WriteLine("=====");
Console.WriteLine(" CineTrack Token Generator");
Console.WriteLine("=====");
Console.WriteLine();

var tokenProvider = TokenProvider.CreateSharedAccessSignatureTokenProvider(keyName, key);

// Gera token com validade de 50 anos
var token = await tokenProvider.GetTokenAsync(uri, TimeSpan.FromDays(365 * 50));

Console.WriteLine("TOKEN GERADO (validade: 50 anos):");
Console.WriteLine("-----");
Console.WriteLine(token.TokenString);
Console.WriteLine("-----");
Console.WriteLine();
Console.WriteLine("Cole este token no CineTrackApiClient.cs:");
Console.WriteLine("private const string SasToken = \"<TOKEN>\";");
```

9. Banco de Dados

Script SQL Completo

```
sql
```

```
-- =====  
-- CineTrack Database - Script Completo  
-- =====
```

```
IF NOT EXISTS (SELECT * FROM sys.databases WHERE name = 'CineTrackDb')
```

```
    CREATE DATABASE CineTrackDb;
```

```
GO
```

```
USE CineTrackDb;
```

```
GO
```

```
-- Tabela de Midias
```

```
CREATE TABLE Midia (
```

```
    Id INT IDENTITY(1,1) PRIMARY KEY,
```

```
    Titulo NVARCHAR(300) NOT NULL,
```

```
    TituloOriginal NVARCHAR(300) NULL,
```

```
    Tipo INT NOT NULL DEFAULT 1,
```

```
    Ano INT NULL,
```

```
    Descricao NVARCHAR(MAX) NULL,
```

```
    Generos NVARCHAR(200) NULL,
```

```
    DuracaoMinutos INT NULL,
```

```
    Elenco NVARCHAR(MAX) NULL,
```

```
    ImagemUrl NVARCHAR(500) NULL,
```

```
    ImagemBase64 NVARCHAR(MAX) NULL,
```

```
    TmdbId INT NULL,
```

```
    ImdbId NVARCHAR(20) NULL,
```

```
    RottenTomatoesUrl NVARCHAR(500) NULL,
```

```
    SceneSourceUrl NVARCHAR(500) NULL,
```

```
    ImdbRating DECIMAL(3,1) NULL,
```

```
    ImdbVotes INT NULL,
```

```
    Tomatometer INT NULL,
```

```
    Popcornmeter INT NULL,
```

```
    Metacritic INT NULL,
```

```
    DataCadastro DATETIME NOT NULL DEFAULT GETDATE(),
```

```
    DataAtualizacao DATETIME NULL,
```

```
    Status INT NOT NULL DEFAULT 1
```

```
);
```

```
CREATE INDEX IX_Midia_TmdbId ON Midia(TmdbId);
```

```
CREATE INDEX IX_Midia_ImdbId ON Midia(ImdbId);
```

```
CREATE INDEX IX_Midia_Tipo ON Midia(Tipo);
```

```
GO
```

```
-- Tabela de Temporadas
```

```
CREATE TABLE Temporada (
```

```
    Id INT IDENTITY(1,1) PRIMARY KEY,
```

```

MidiaId INT NOT NULL FOREIGN KEY REFERENCES Midia(Id) ON DELETE CASCADE,
Numero INT NOT NULL,
Titulo NVARCHAR(200) NULL,
Ano INT NULL,
TotalEpisodios INT NOT NULL DEFAULT 0,
Descricao NVARCHAR(MAX) NULL,
ImagemUrl NVARCHAR(500) NULL,
DataCadastro DATETIME NOT NULL DEFAULT GETDATE(),
Status INT NOT NULL DEFAULT 1,
CONSTRAINT UK_Temporada UNIQUE (MidiaId, Numero)
);
GO

-- Tabela de Episódios
CREATE TABLE Episodio (
    Id INT IDENTITY(1,1) PRIMARY KEY,
    TemporadaId INT NOT NULL FOREIGN KEY REFERENCES Temporada(Id) ON DELETE CASCADE,
    Numero INT NOT NULL,
    Titulo NVARCHAR(300) NULL,
    Descricao NVARCHAR(MAX) NULL,
    DuracaoMinutos INT NULL,
    DataLancamento DATE NULL,
    ImagemUrl NVARCHAR(500) NULL,
    ImdbRating DECIMAL(3,1) NULL,
    DataCadastro DATETIME NOT NULL DEFAULT GETDATE(),
    Status INT NOT NULL DEFAULT 1,
    CONSTRAINT UK_Episodio UNIQUE (TemporadaId, Numero)
);
GO

-- Status do usuário para mídia
CREATE TABLE UsuarioMidia (
    Id INT IDENTITY(1,1) PRIMARY KEY,
    MidiaId INT NOT NULL FOREIGN KEY REFERENCES Midia(Id) ON DELETE CASCADE,
    Status INT NOT NULL DEFAULT 3,
    AssistindoCom INT NULL,
    NotaUsuario DECIMAL(3,1) NULL,
    EstrelaUsuario INT NULL,
    Comentario NVARCHAR(MAX) NULL,
    DataInicio DATETIME NULL,
    DataConclusao DATETIME NULL,
    DataCadastro DATETIME NOT NULL DEFAULT GETDATE(),
    DataAtualizacao DATETIME NULL,
    CONSTRAINT UK_UsuarioMidia UNIQUE (MidiaId)
);
GO

```

```
-- Status do usuário para episódio
```

```
CREATE TABLE UsuarioEpisodio (  
  Id INT IDENTITY(1,1) PRIMARY KEY,  
  EpisodioId INT NOT NULL FOREIGN KEY REFERENCES Episodio(Id) ON DELETE CASCADE,  
  Status INT NOT NULL DEFAULT 2,  
  DataAssistido DATETIME NULL,  
  AssistindoCom INT NULL,  
  Comentario NVARCHAR(MAX) NULL,  
  DataCadastro DATETIME NOT NULL DEFAULT GETDATE(),  
  CONSTRAINT UK_UsuarioEpisodio UNIQUE (EpisodioId)  
);  
GO
```

```
-- Associações pendentes
```

```
CREATE TABLE AssociacaoPendente (  
  Id INT IDENTITY(1,1) PRIMARY KEY,  
  TituloOriginal NVARCHAR(500) NOT NULL,  
  TituloSugerido NVARCHAR(500) NOT NULL,  
  TmdbId INT NULL,  
  Confianca DECIMAL(5,2) NOT NULL,  
  Status INT NOT NULL DEFAULT 0,  
  SceneSourceUrl NVARCHAR(500) NULL,  
  DataCaptura DATETIME NOT NULL DEFAULT GETDATE(),  
  DataProcessamento DATETIME NULL  
);  
GO  
  
PRINT 'CineTrack Database criado com sucesso!';  
GO
```

10. Roteiro de Implantação

Passo 1: Banco de Dados

```
powershell
```

```
# Execute o script SQL no SQL Server Management Studio
```

Passo 2: Azure Relay

Veja seção 11 para detalhes completos.

Passo 3: API

```
powershell
```

```
cd CineTrack.API
dotnet publish -c Release -o C:\CineTrack\API
```

Passo 4: RelayListener

```
powershell

cd CineTrackRelayListener
dotnet publish -c Release -r win-x64 --self-contained -o C:\CineTrack\RelayListener
cd C:\CineTrack\RelayListener
.install-service.bat # Executar como Admin
```

Passo 5: Gerar Token

```
powershell

cd CineTrackTokenGen
dotnet run
# Copie o token gerado para CineTrackApiClient.cs
```

Passo 6: App Mobile

```
powershell

cd CineTrack.Mobile
dotnet build -t:Run -f net8.0-android
```

11. Configuração do Azure Relay - Passo a Passo Detalhado

11.1 Acessar Portal Azure

1. Abra: <https://portal.azure.com>
2. Faça login

11.2 Criar Relay Namespace

1. Pesquise "**Relay**" na barra superior
2. Clique "**Relays**"
3. Clique "+ Create"
4. Preencha:
 - **Subscription:** Sua assinatura
 - **Resource Group:** rg-cinetrack (criar novo)

- **Namespace name:** `cinetrack-relay`
- **Location:** `Brazil South`

5. Clique **"Review + Create"** → **"Create"**
6. Aguarde e clique **"Go to resource"**

11.3 Criar Hybrid Connection

1. Menu lateral: **"Hybrid Connections"**
2. Clique **" + Hybrid Connection "**
3. **Name:** `cinetrack-bridge`
4. Marque **"Requires Client Authorization"**
5. Clique **"Create"**

11.4 Criar Política SAS

1. Clique em **"cinetrack-bridge"**
2. Menu: **"Shared access policies"**
3. Clique **" + Add "**
4. **Policy name:** `ListenSend`
5. Marque: **Send** ☒ e **Listen** ☒
6. Clique **"Create"**

11.5 Copiar Connection String

1. Clique na política **"ListenSend"**
2. Copie **"Connection string – Primary key"**

11.6 Verificar Conectividade

Após instalar o RelayListener, veja em:

- **Relay > Hybrid Connections > cinetrack-bridge > Listeners**
 - Deve mostrar **"1"** se conectado
-

12. Troubleshooting

Erro: "TokenMissingOrInvalid"

- Verifique se o token foi gerado com `https://` (não `http://`)
- Regenere o token

Erro: "404 Not Found" via Relay

- Verifique se RelayListener está rodando: `Get-Service CineTrackRelayListener`
- Verifique se API está rodando: `Invoke-RestMethod http://localhost:5000/health`

App não carrega dados

- Verifique conectividade com internet
 - Verifique se o token está correto no CineTrackApiClient.cs
-

Conclusão

O **CineTrack** é um sistema completo para gerenciamento pessoal de filmes e séries, com:

- ☒ Scraping automático de novos lançamentos
 - ☒ Matching inteligente com algoritmos Levenshtein + Jaccard
 - ☒ 4 fontes de ratings integradas
 - ☒ Acesso remoto seguro via Azure Relay
 - ☒ App mobile nativo com .NET MAUI
-

Documento gerado com base nas conversas de desenvolvimento do projeto CineTrack. Última atualização: Dezembro 2025